EE5239 Introduction to Nonlinear Optimization

Lecture 5: Case Study: Logistic Regression

Mingyi Hong and Jiaxiang Li

University Of Minnesota

Today

- **Today**: (Theory-Guided) Case study of logistic regression (LR), mostly using what you have learned
  - Note: examples may be (much) more complicated than theory
- After today's lecture, you will be able to
  - identify a few optimization issues of LR
  - explain the algorithm behavior of simple LR using existing theory
- More advanced goal
  - learn how to analyze an optimization problem step-by-step
  - apply this analysis framework to other complicated problems (e.g. in projects)

Today

- HW 3 will be out soon
- Hw 2 solution will be out soon
- Proposal due next week [I won't grade the proposal, but make comments; If late, OK]

Outline

Review of Theories You Learned

- Optimality conditions: check local-min/global-min
  - Convexity is crucial
- When does GD "converge"?
  - Stepsize choice: $< 2/L$ with L-Lipschitz gradient
  - line search
- Convergence rate? Depending on condition number for strongly convex problems
- Other methods, coordinates descent, incremental method, many more

Apply Theory?

- We will see: it is nontrivial to apply these theories to even a simple problem

- From now on, imagine yourself being a data scientist/engineer/etc., trying to solve a problem.
  - Step 1: Specify the problem
  - Step 2: Write or modify code to solve it

Apply Theory?

- We will see: it is nontrivial to apply these theories to even a simple problem

- From now on, imagine yourself being a data scientist/engineer/etc., trying to solve a problem.
  - Step 1: Specify the problem
  - Step 2: Write or modify code to solve it
  - Step 3: Observe algorithm behavior, good? bad? expected or not?
  - Step 4: If not good/expected, go to Step 2

Apply Theory?

- We will see: it is nontrivial to apply these theories to even a simple problem
- From now on, imagine yourself being a data scientist/engineer/etc., trying to solve a problem.
  - Step 1: Specify the problem
  - Step 2: Write or modify code to solve it
  - Step 3: Observe algorithm behavior, good? bad? expected or not?
  - Step 4: If not good/expected, go to Step 2
- Keep in mind: as an optimizer, your goal is to understand all behavior (if possible)
  - The goal of a regular data scientist is probably "making it work"

Warm-up: 1-dim linear regression

Note that from now on

- $\mathbf{w}$ represents our optimization variable
- $\{\mathbf{x}_i, y_i\}$ are the data sets

Warm-up: 1-dim linear regression

Before discussing logistic regression, we do a warm-up for linear regression.

- Set a baseline for the behavior
- Try to understand everything you observe

Problem setup: $\min_w \frac{1}{2}(y - x \cdot w)^2$.

- Problem parameters: $y = 1, x = 25$.

Algorithm: $w^+ = w - \alpha \nabla f(w)$.

- Initialization: $w^0 = 5$. (or randn(1,1); let's do fixed first)
- Max-iteration: MaxIte $= 50$. (can change)

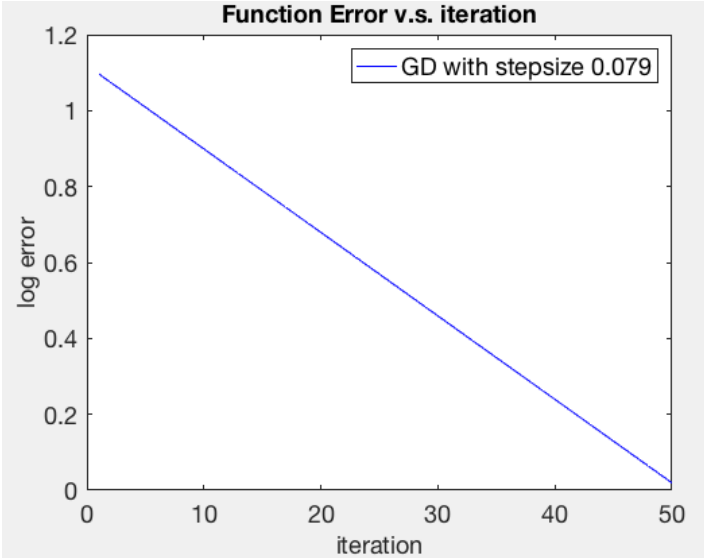
Tuning parameter: stepsize α.

1-dim regression: threshold

Start from

- $\alpha = 1$, diverge; $\alpha = 0.1$, diverge; $\alpha = 0.01$, converge.

You want to know the threshold, and find the threshold is 0.08.



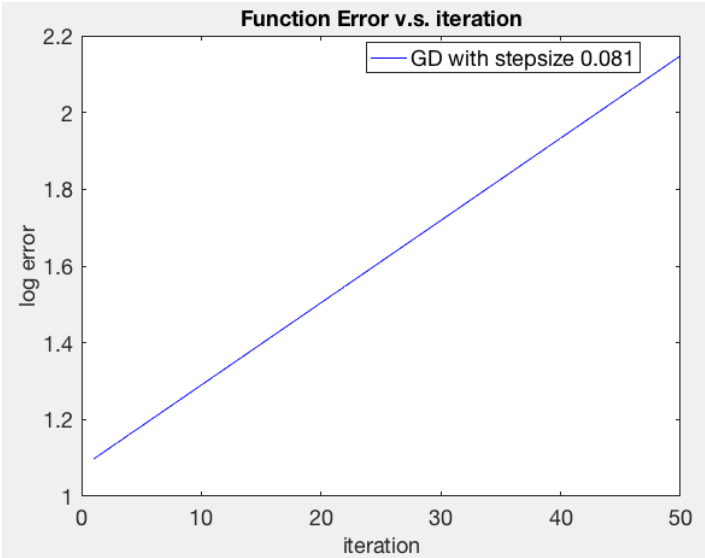
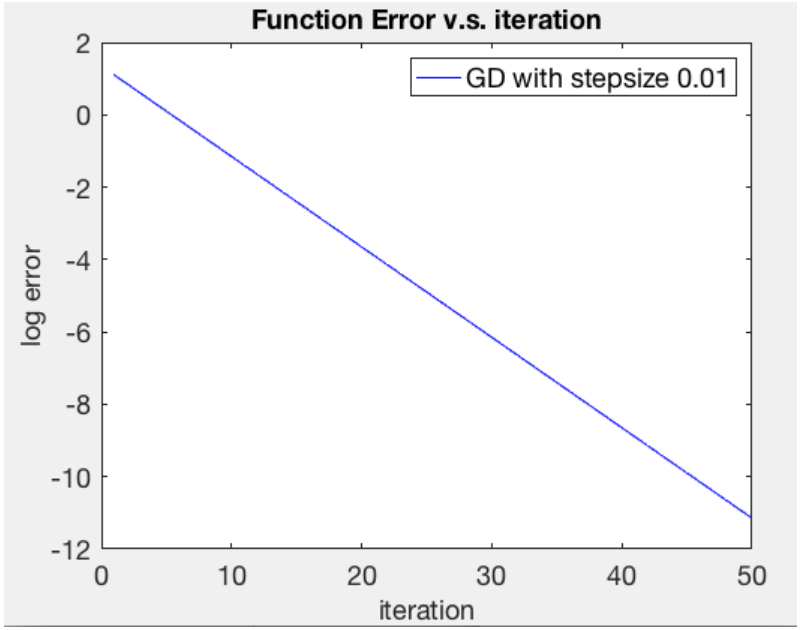


Figure 0.1: Left: stepsize 0.081; Right: stepsize 0.079

By theory, the threshold of $\alpha = 2/L = 2/x^2 = 0.08$.

1-dim regression: speed

You want to find α so it converges fast.



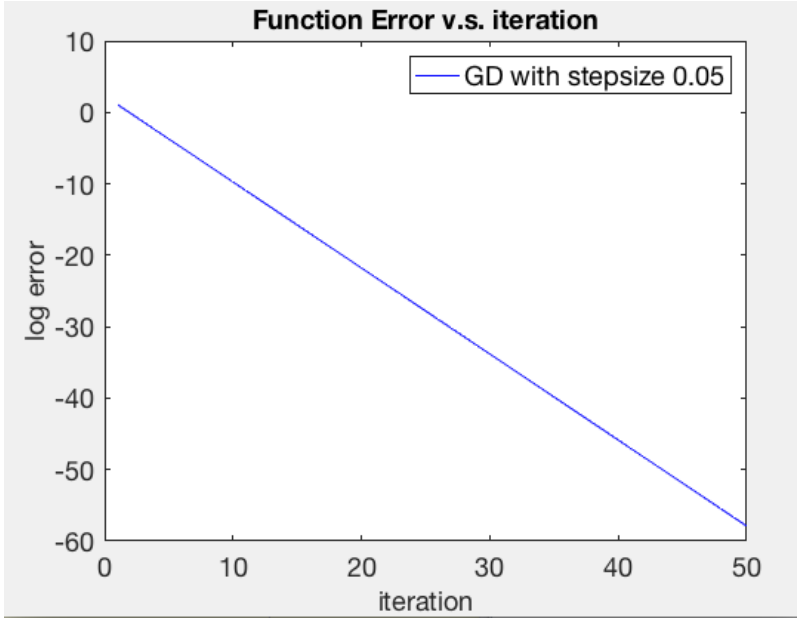


Figure 0.2: Left: stepsize 0.01; Right: stepsize 0.05

Stepsize affects the convergence speed a lot.

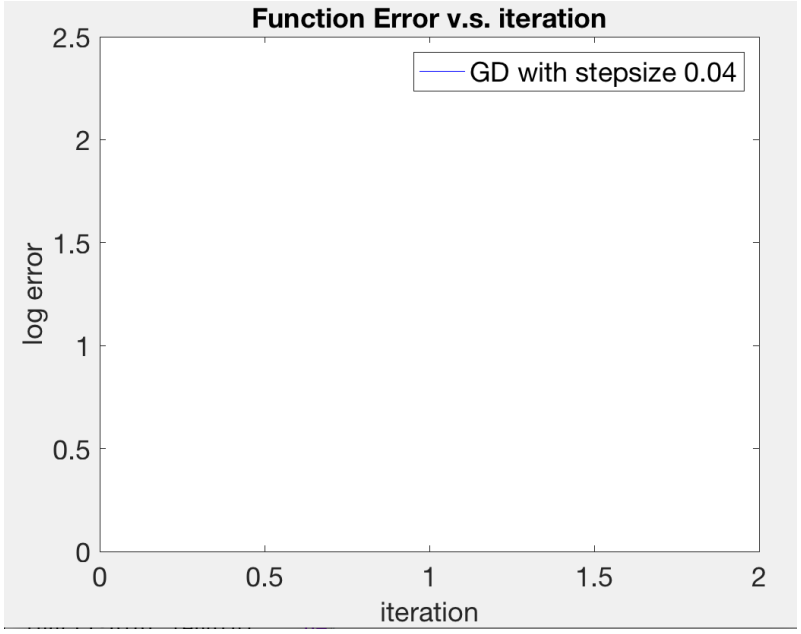
$\alpha = 0.01$, to achieve 10^{-10}, need _____ iterations.

$\alpha = 0.05$, to achieve 10^{-10}, need _____ iterations.

What is the optimal stepsize?

1-dim regression: best stepsize

Let's check $\alpha = 0.04 = 1/L$ and $\alpha = 0.08 = 2/L$.



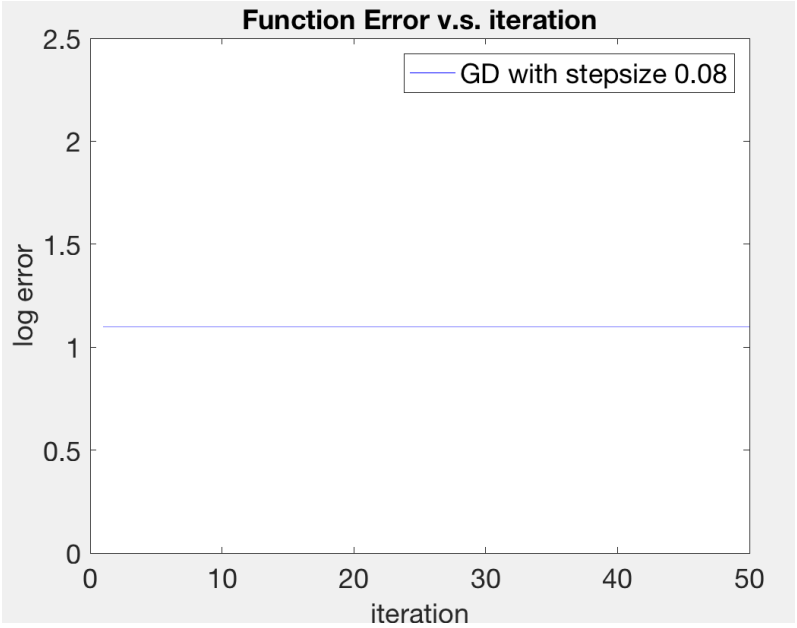


Figure 0.3: Left: stepsize 0.04; Right: stepsize 0.08

$\alpha = 0.04$: algorithm converges too fast.

Check numerical values: $\log_{10} f(x^k) = 1.069$, -Inf, -Inf, ...

1-dim regression: summary

Summary of 1-dim linear regression: stepsize affects convergence speed.

For convergence, two phases:

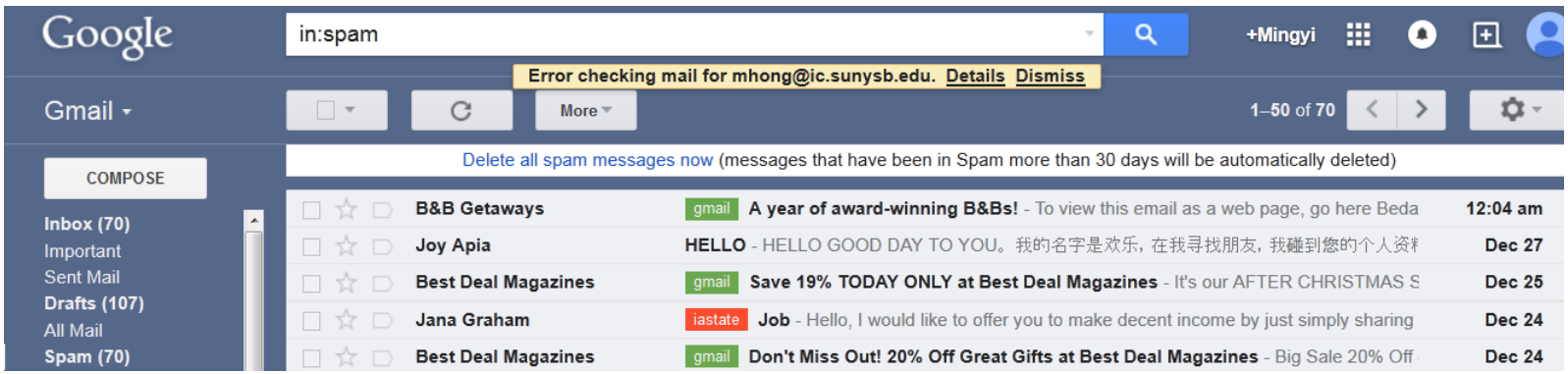
- Phase 1: $\alpha > 0.08 = 2/L$. Diverge.
- Phase 2: $0 < \alpha < 0.08 = 2/L$. Converge.

For convergence speed, two phases:

- Phase 1: $0 < \alpha < 0.04$ bigger stepsize implies faster
- Phase 2: $0.04 < \alpha < 0.08 = 2/L$, bigger stepsize implies slower.

Classification Problem

- Classification: one of the key problems of machine learning
- Example: Spam email.
- Every email services have automatic spam detection mechanisms
- The basic task is the following
  1. Given an excerpt of an email, represented by a vector $\mathbf{x}_i$
  2. Ask the question whether it is a spam or not ($y_i = +1, -1$)



Example: Image Classification

- More complicated examples: image classification

ImageNet Challenge

IMAGENET

- 1,000 object classes (categories).
- Images:
  - 1.2 M train
  - 100k test.

4

Linear Classification

Regression: $(x_i, y_i), i = 1, \ldots, n$.

- Find w such that dist$(w^T x_i, y_i)$ is small.
- Choose square loss function [remember this, HW 3 will use this]

$$\min_w \sum_i \text{dist}(w^T x_i, y_i) = \sum_i \|w^T x_i - y_i\|^2.$$

Classification: $(x_i, y_i), i = 1, \ldots, n$, where $y_i \in \{1, -1\}$.

- Find w such that dist$(w^T x_i, y_i)$ is small
- First, choose 0-1 loss function (sign of $w^T x_i$ and y_i are the same)

$$\min_w \sum_i \text{dist}(w^T x_i, y_i) = \sum_i \text{sign}(-y_i w^T x_i).$$

- Second, choose a loss function $\ln(1 + \exp(-y\hat{y}))$

$$\min_w \sum_i \text{dist}(w^T x_i, y_i) = \sum_i \ln(1 + \exp(-y_i w^T x_i)).$$

Comparison of Two Loss Functions

Problem: determine the "distance" between the prediction $\hat{y} = w^T x$ and true label $y \in \{1, -1\}$.

Ideal $0-1$ loss function (as a function of $y\hat{y}$):

Surrogate loss function $\ln(1 + \exp(-z))$ where $z = y\hat{y}$

Logistic Regression: Detailed formulation (side)

- **Question:** Where does this objective come from?
- It is used to predict probability for certain events

Logistic Regression: Detailed formulation (side)

- **Question:** Where does this objective come from?
- It is used to predict probability for certain events
- Recall in the previous models, predict a numerical quantity using $\mathbf{x}^T\mathbf{w}$, and predict $(-1, 1)$ using $\text{sgn}(\mathbf{x}^T\mathbf{w})$

Logistic Regression: Detailed formulation (side)

- **Question:** Where does this objective come from?
- It is used to predict probability for certain events
- Recall in the previous models, predict a numerical quantity using $\mathbf{x}^T\mathbf{w}$, and predict $(-1, 1)$ using $\text{sgn}(\mathbf{x}^T\mathbf{w})$
- A new model will be used here to predict probability

$$h(\mathbf{x}) = \theta(\mathbf{x}^T\mathbf{w})$$

- Here the function $\theta(\cdot)$ is the so-called logistic function:

$$\theta(s) = \frac{e^s}{1+e^s} \in (0, 1)$$

Logistic Regression: Detailed formulation (side)

- Here the learning objective is the probability

$$h(\mathbf{x}) = P(y = 1 \mid \mathbf{x})$$

- For a data point, what's the probability that it is of class "+1"

Logistic Regression: Detailed formulation (side)

- Here the learning objective is the probability

$$h(\mathbf{x}) = P(y = 1 \mid \mathbf{x})$$

- For a data point, what's the probability that it is of class "+1"
- So the posterior distribution is given by

$$P(y \mid \mathbf{x}) = \begin{cases} h(\mathbf{x}) & \text{for } y = +1; \\ 1 - h(\mathbf{x}) & \text{for } y = -1. \end{cases}$$

- Let us use the logistic function to model $h(\mathbf{x})$ (make sense?)

$$h(\mathbf{x}) = \frac{e^{\mathbf{x}^T \mathbf{w}}}{1 + e^{\mathbf{x}^T \mathbf{w}}}$$

Logistic Regression: Detailed formulation (side)

- The posterior distribution is given by

$$P(y \mid \mathbf{x}) = \begin{cases} \frac{e^{\mathbf{x}^T \mathbf{w}}}{1+e^{\mathbf{x}^T \mathbf{w}}} = \theta(\mathbf{x}^T \mathbf{w}) & \text{for } y = +1; \\ 1 - \frac{e^{\mathbf{x}^T \mathbf{w}}}{1+e^{\mathbf{x}^T \mathbf{w}}} = 1 - \theta(\mathbf{x}^T \mathbf{w}) & \text{for } y = -1. \end{cases}$$

Logistic Regression: Detailed formulation (side)

- The posterior distribution is given by

$$P(y \mid \mathbf{x}) = \begin{cases} \frac{e^{\mathbf{x}^T \mathbf{w}}}{1+e^{\mathbf{x}^T \mathbf{w}}} = \theta(\mathbf{x}^T \mathbf{w}) & \text{for } y = +1; \\ 1 - \frac{e^{\mathbf{x}^T \mathbf{w}}}{1+e^{\mathbf{x}^T \mathbf{w}}} = 1 - \theta(\mathbf{x}^T \mathbf{w}) & \text{for } y = -1. \end{cases}$$

- Write the above expression in a compact form?

Logistic Regression: Detailed formulation (side)

- The posterior distribution is given by

$$P(y \mid \mathbf{x}) = \begin{cases} \frac{e^{\mathbf{x}^T \mathbf{w}}}{1+e^{\mathbf{x}^T \mathbf{w}}} = \theta(\mathbf{x}^T \mathbf{w}) & \text{for } y = +1; \\ 1 - \frac{e^{\mathbf{x}^T \mathbf{w}}}{1+e^{\mathbf{x}^T \mathbf{w}}} = 1 - \theta(\mathbf{x}^T \mathbf{w}) & \text{for } y = -1. \end{cases}$$

- Write the above expression in a compact form?
- Use the key fact that $1 - \theta(s) = \theta(-s)$ [verify!]

$$P(y \mid \mathbf{x}) = \theta(y \mathbf{x}^T \mathbf{w})$$

- Data points are independently generated, then the probability of getting $y_1, y_2, \cdots, y_n$ becomes

$$P(y_1, \cdots, y_n \mid \mathbf{x}_1, \cdots, \mathbf{x}_n) = \prod_{i=1}^{n} P(y_i \mid \mathbf{x}_i)$$

Logistic Regression: Detailed formulation (side)

- Make the optimal decision by maximizing the joint posterior probability

$$\frac{1}{n} \ln \left(\prod_{i=1}^{n} P(y_i \mid \mathbf{x}_i) \right)$$

- Or equivalently minimizing

$$\frac{1}{n} \ln \left(\prod_{i=1}^{n} \frac{1}{P(y_i \mid \mathbf{x}_i)} \right) = \frac{1}{n} \ln \left(\prod_{i=1}^{n} \frac{1}{\theta(y_i \mathbf{x}_i^T \mathbf{w})} \right)$$

- Now plug in the logistic function $\theta(\cdot)$, we see that the "loss/error" function for logistic regression becomes

$$L(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^{n} \ln \left(1 + e^{-y_i \mathbf{x}_i^T \mathbf{w}} \right)$$

Side: Popularity of Logistic Regression

Google search "logistic regression": about 10 million results.

"There is a perception that LR is slow, unstable, and unsuitable for large learning or classification tasks" –Komarak'04 "Logistic Regression for Data Mining and High- Dimensional Classification"

But It is still very popular nowadays:

- deep learning is built on LR
- very commonly used in IT companies like Facebook, Google, etc.

There are many tutorials on LR.

- One is Stanford CS231n course webpage
- Or you can check a detailed tutorial at http://www.dataschool.io/guide-to-logistic-regression/

We focus on optimization issues in this course.

Simplest problem: 1-dim case

- **Problem setup**:
  - Two data points in $\mathbb{R}$, x_1 and x_2.
  - Labels $y_1 = 1$, $y_2 = -1$.

- Remark: typically, one would use $w_1 \cdot x_i + w_2$ to classify; but for simplicity, let's use $w \cdot x_i$ for now (setting the intercept $w_2 = 0$).

- Question: What are the models are we considering?

- **Objective function** (I have plugged in y_1 and y_2)

$$f(w) = \frac{1}{2}\log(1 + \exp(-w \cdot x_1)) + \frac{1}{2}\log(1 + \exp(w \cdot x_2)).$$

Algorithm: $x^+ = x - \alpha \nabla f(x)$.

  - Initialization: $w^0 = 5$. (or randn(1,1); let's do fixed first)
  - Max-iteration: MaxIte $= 20$. (can change)

Tuning parameter: stepsize α.

Preliminary Analysis

Before running the algorithm, let's do some simple calculations.

- First, is the problem convex?

Preliminary Analysis

Before running the algorithm, let's do some simple calculations.

- First, is the problem convex?

- Obtain $f''(w) = (x_1)^2 \frac{e^{wx_1}}{(1+e^{wx_1})^2} + (x_2)^2 \frac{e^{wx_2}}{(1+e^{wx_2})^2} > 0.$

Preliminary Analysis

Before running the algorithm, let's do some simple calculations.

- First, is the problem convex?

- Obtain $f''(w) = (x_1)^2 \frac{e^{wx_1}}{(1+e^{wx_1})^2} + (x_2)^2 \frac{e^{wx_2}}{(1+e^{wx_2})^2} > 0.$

- Second, is the problem strongly convex?

Preliminary Analysis

Before running the algorithm, let's do some simple calculations.

- First, is the problem convex?
- Obtain $f''(w) = (x_1)^2 \frac{e^{wx_1}}{(1+e^{wx_1})^2} + (x_2)^2 \frac{e^{wx_2}}{(1+e^{wx_2})^2} > 0.$
- Second, is the problem strongly convex?
- No, it is strictly convex (why?)

First experiment

Data Setting 1: Set $x_1 = 1, x_2 = 2$.

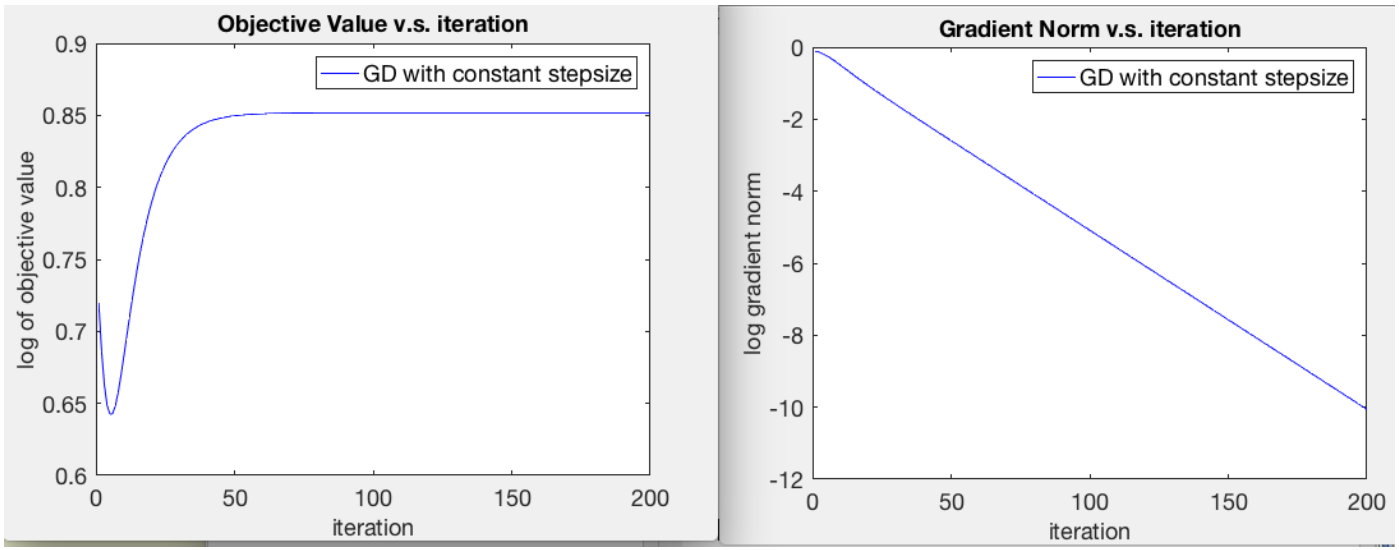


Figure 2.1: Function Value. $\alpha = 0.2$ for 1-dim logistic regression

- Why increasing?

First experiment

Data Setting 1: Set $x_1 = 1, x_2 = 2$.

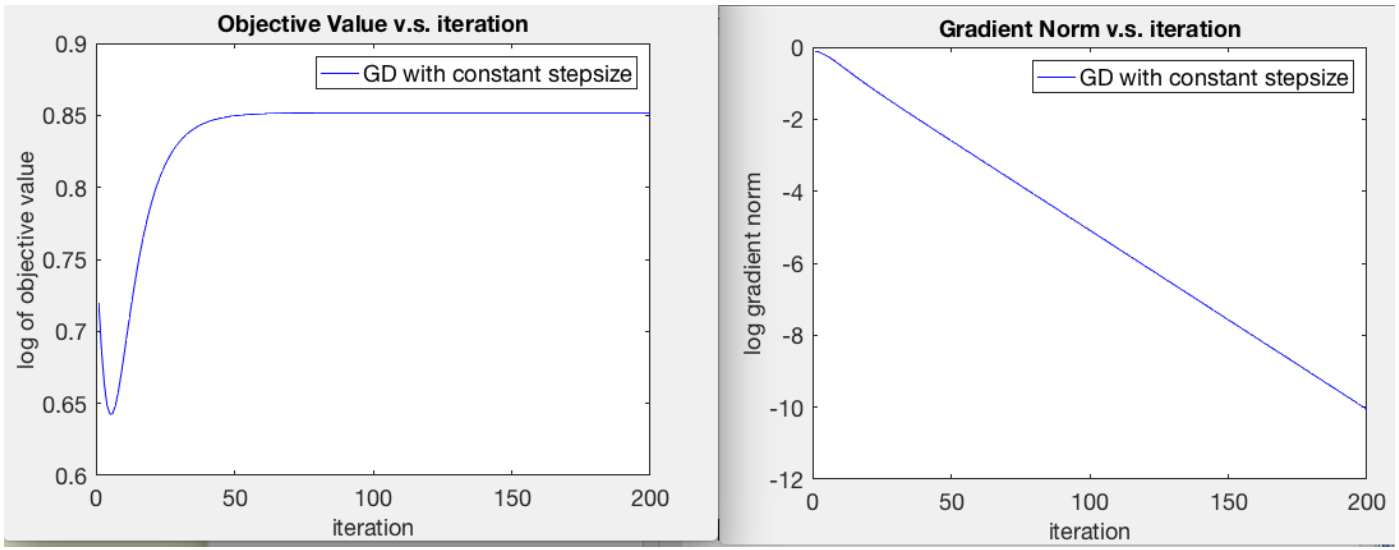


Figure 2.1: Function Value. $\alpha = 0.2$ for 1-dim logistic regression

- Why increasing?
- BUG! Used $\log(1 + \exp(y_i w^T x_i))$, missing a " $-$ ".

First experiment

Data Setting 1: Set $x_1 = 1, x_2 = 2$.

Pick $\alpha = 0.2$. Figure of the algorithm for 200 iterations:

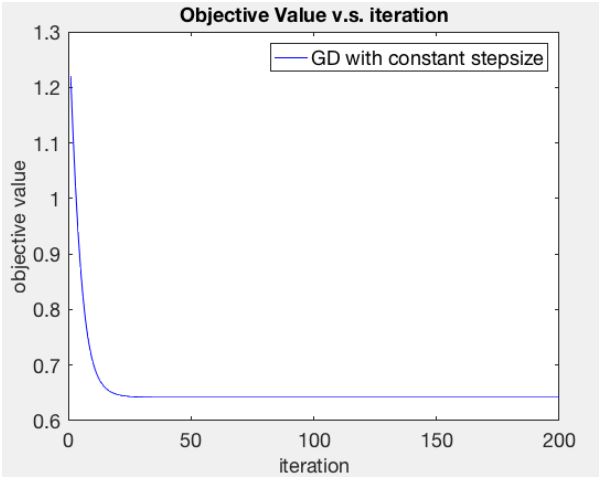


Figure 2.2: Function Value. $\alpha = 0.2$ for 1-dim logistic regression

Question: Has it already converged?

Issue: in theory, use $f(w^r) - f^*$ to measure error,

but in practice don't know _____

Performance Metric and Stopping Criteria

One choice: $f(w^r) - f(w^{r-1})$; last few iterations of $f(w^r)$:

$0.6420, 0.6420, 0.6420, \ldots$

Drawbacks

- cannot see from figure;
- what meaning?

A better choice: gradient norm $\|\nabla f(w^r)\|$

Performance Metric and Stopping Criteria

One choice: $f(w^r) - f(w^{r-1})$; last few iterations of $f(w^r)$:

$0.6420, 0.6420, 0.6420, \ldots$

Drawbacks

- cannot see from figure;
- what meaning?

A better choice: gradient norm $\|\nabla f(w^r)\|$

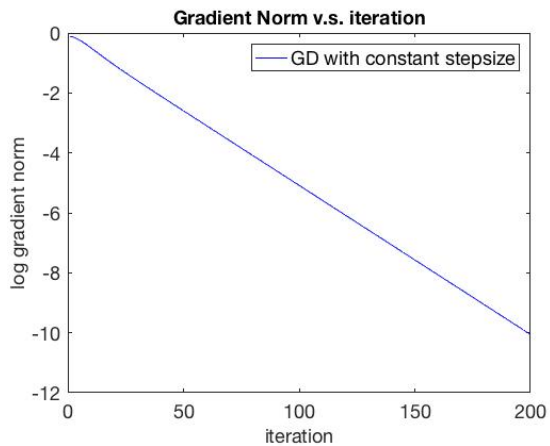


Figure 2.3: Gradient norm. $\alpha = 0.2$ for 1-dim logistic regression

Stopping Criteria

Two metric: $\nabla f(x^r)$ and $|f(x^r) - f(x^{r+1})|$ (the later should not be used when diminishing stepsize is used).

Can stop the algorithm when either of the two holds:

- when $\|\nabla f(w^r)\| \leq \epsilon$, or $\|\nabla f(w^r)\|/\|\nabla f(w^0)\| \leq \epsilon$
- when $|f(w^r) - f(w^{r+1})| \leq \epsilon$
- when $\|w^r - w^{r+1}\| \leq \epsilon$

Typical choice of error: $\epsilon = 10^{-3}$ (low precision), or $\epsilon = 10^{-10}$ (high precision)

Practice: relation between the three criteria?

Second experiment

Data Setting 2: Set $x_1 = 1, x_2 = -2$.

Pick $\alpha = 0.2$. Figure of the algorithm for 200 iterations (use log scale):

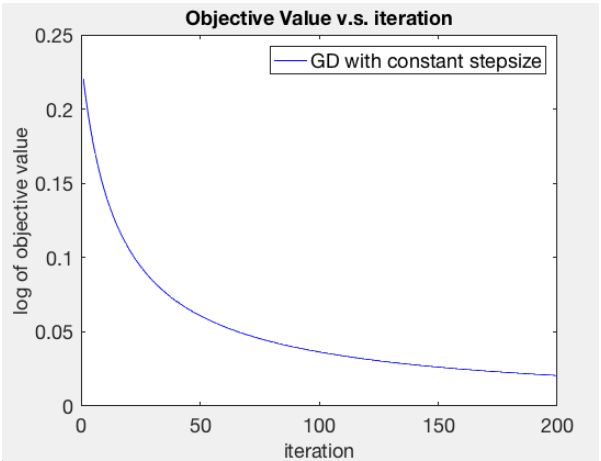


Figure 3.1: Function Value. $\alpha = 0.2$ for 1-dim logistic regression

Question: Has it "converged"?

Converged or Not?

Let's run 20k iterations. That should be enough, uhh?

Check log of function values, and gradient norms

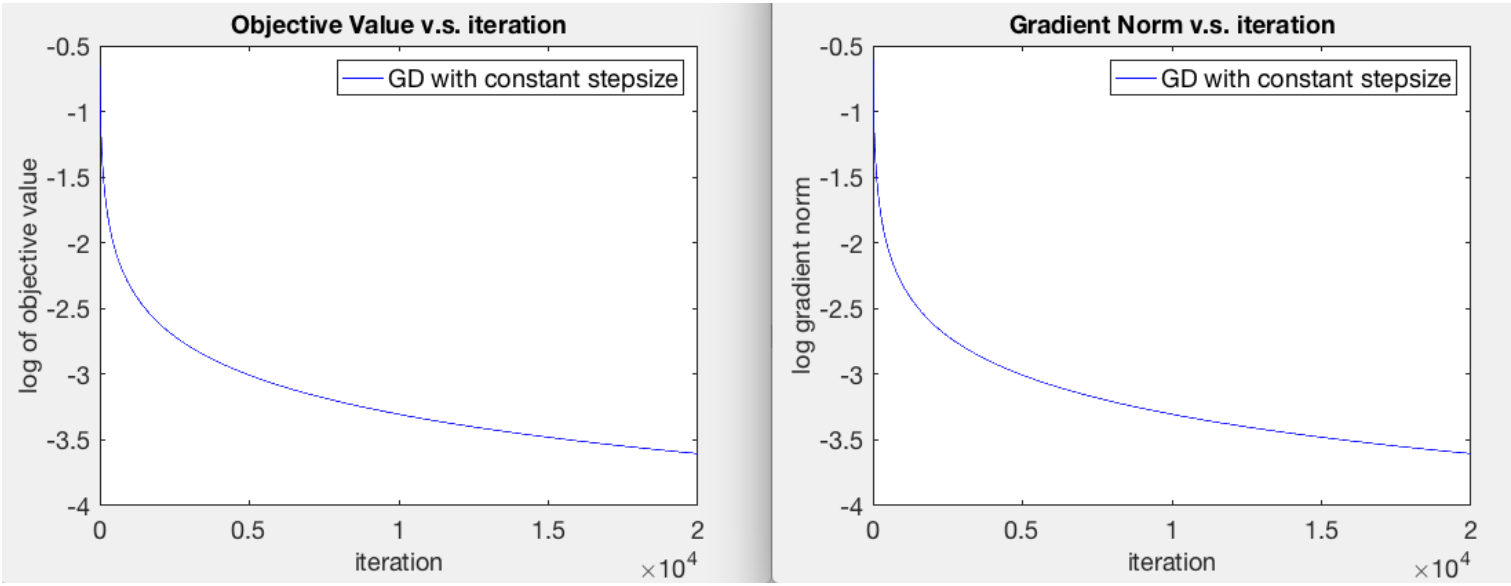


Figure 3.2: L: Log of Function Value. R: Gradient Norm. 20k iterations

Again, has it "converged"? Or, should we count it as "converged"?

- Yes, since the loss function is small?
- No, since the gradient norm is still not small enough?

Discussion: Why This Happens

So strange... In all previous cases (linear regression, or Data Setting 1), either diverge or converge in the sense that "error" $< 10^{-10}$.

Re-examine the theory

- If $\alpha < 2/L$, then converge to stationary point.
- If convex, then converge to globally-optimal.

So, it should finally converge to a stationary point, which is globally optimal (right?). Maybe 20,000 iterations is not enough?

First Issue

First issue: Non-existence of global-min.

- Recall the graph of $\log(1+\exp(-z))$, where $z = y\hat{y}$.

- **Observation**: Although strictly convex and lower bounded, it has no stationary point.

- Iterates diverge: $\|w^k\|$ goes to ∞ to minimize $f(w)$

Second Issue

- **Second Issue**: Convergence.
- Does the algorithm still converges?
- But it seems that function values are converging to $0 = f^*$.

Second Issue

- **Second Issue**: Convergence.
- Does the algorithm still converges?
- But it seems that function values are converging to $0 = f^*$.
- **Claim:** Using GD with constant stepsize $\alpha < 2/L$ for this 1-dim LR problem, we have $f(w^r) \to f^* = \inf_w f(w)$.

Third Issue

- **Third Issue**: Convergence speed.
- Now we know $f(w^r)$ does converge to 0. But why is it so slow?
- First thought? Bug.

  Why do I insist on explaining this phenomenon?

Third Issue: convergence speed

- Second thought: condition number too large.
- Compute condition number at every point, turn out to be... 1.
  - 1-dim problem, certainly at each point $\lambda_{\max} = \lambda_{\min}$
- What's wrong?

Third Issue: convergence speed

We compute $L(w^r)$ and $\sigma(w^r)$ at all iterates (the largest and smallest eigenvalue of Hessian):

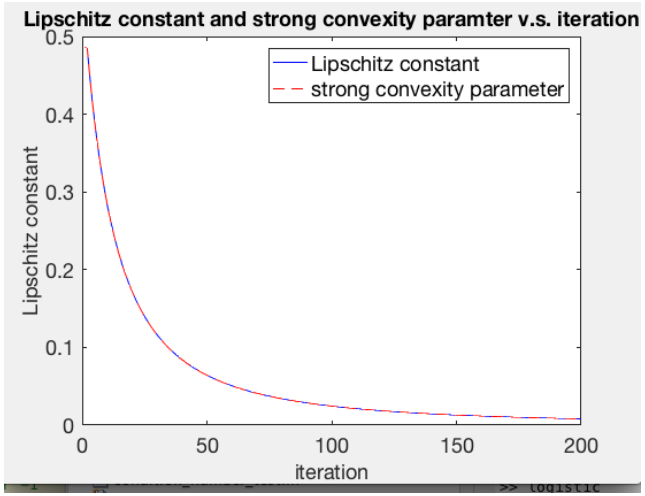


Figure 3.3: L and σ of various iterates

Observation: $L(w^r) = \sigma(w^r)$ is decreasing.

Def. of condition number: $\sigma \leq f''(w) \leq L$ for all w, then $\kappa = \frac{L}{\sigma}$.

E.g. $L(w^1) = 0.5$, $L(w^{200}) \approx 0.008$, then κ is at least $0.5/0.008 = 62.5$.

Non-strongly convex

Theoretically speaking, the condition number of f is $+\infty$.

So, the convergence rate result on κ does not apply here!

Then can we have convergence speed guarantee?

- for f with Lipschitz gradient, not strongly convex
- More general, for the following f (LI here means a diagonal matrix with all diagonal values as "L"):

$$0 \preceq \nabla f(w) \preceq L \times I.$$

Non-strongly-convex Case

Recall the following result in Lecture 3c.

We mentioned that when the problem is convex, but not strongly convex, GD converges with a sublinear rate

Claim: Suppose f is a convex function with Lipschitz gradient. Consider

$$\min_{\mathbf{w}\in\mathbb{R}^n} f(\mathbf{w}).$$

Suppose GD with stepsize $1/L$ generates a sequence $\mathbf{x}^r$, then

$$e^r = f(\mathbf{w}^r) - f(\mathbf{w}^*) \leq \frac{2L}{r}\|\mathbf{w}^0 - \mathbf{w}^*\|^2.$$

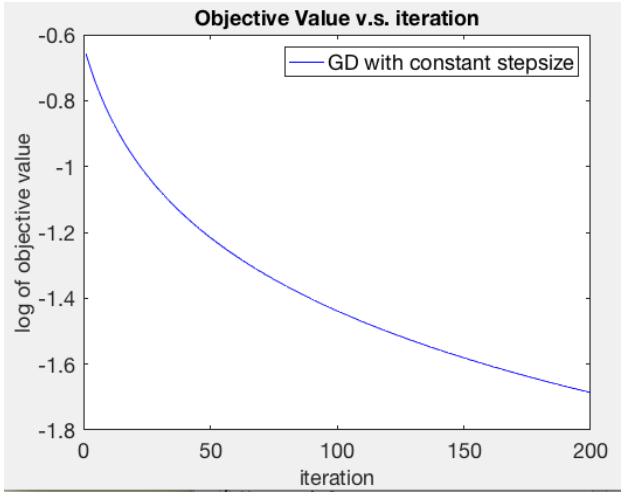
- **Remark:** This is called "sublinear rate" $O(1/r)$.

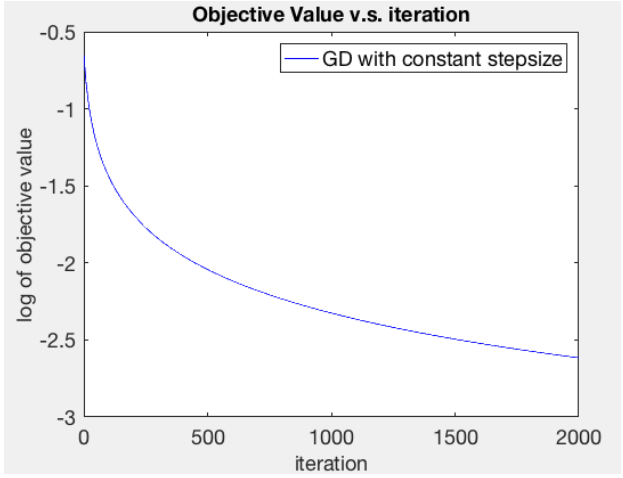
  Linear rate: $1, 0.1, 0.01, \ldots$

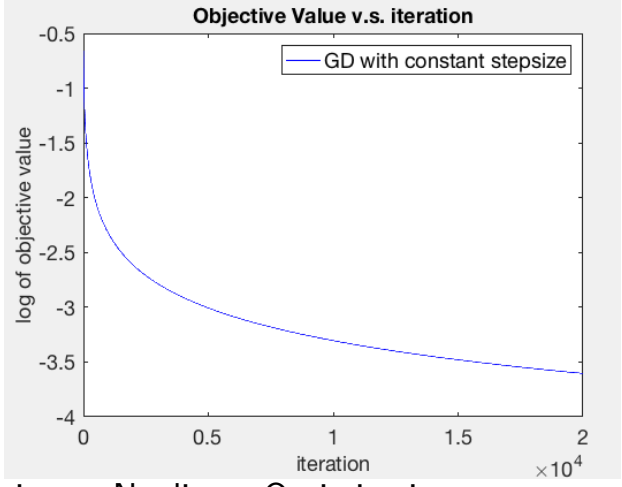
  Sublinear rate: $1, 1/2, 1/3, \ldots, 1/10, \ldots, ...$

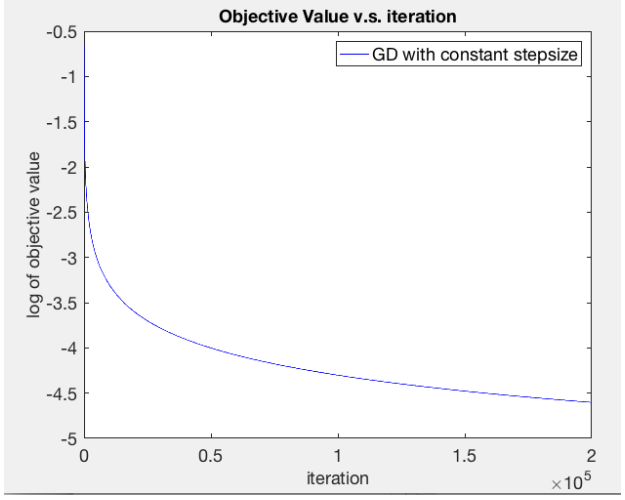
Verifying Sublinear Rate in Practice

Run for 200, 2k, 20k, 200k iterations.









Verifying Sublinear Rate in Practice (cont'd)

- Collect the simulation results in a table:

# of Iterations	200	2k	20k	200k
error	$10^{-1.6}$	$10^{-2.6}$	$10^{-3.6}$	$10^{-4.6}$

Table 1: Error v.s. Iterations

- **Observation**: To reduce error to $1/10$, need 10 times more iterations.
- **Theory**: error is $\frac{1}{r}C$, where $C = 2L\|\mathbf{w}^0 - \hat{\mathbf{w}}\|^2$.
- So the performance (almost) matches the theory! (finally)

Why Two Data Settings Different

Why are Data Setting 1 $(1, 2)$ and Setting 2 $(1, -2)$ so different?

What if we use $(1, 5)$? or $(-0.3, -6)$?

Why Two Data Settings Different

Why are Data Setting 1 $(1, 2)$ and Setting 2 $(1, -2)$ so different?

What if we use $(1, 5)$? or $(-0.3, -6)$?

Two typical data settings:

- Separable case
- Non-separable case
- In separable case, w diverges, arrive at flat region!
- In non-separable case, w stays bounded, so essentially"strongly convex"!

Question: Why care about iterates divergence?

- Because it may affect convergence rate a lot!

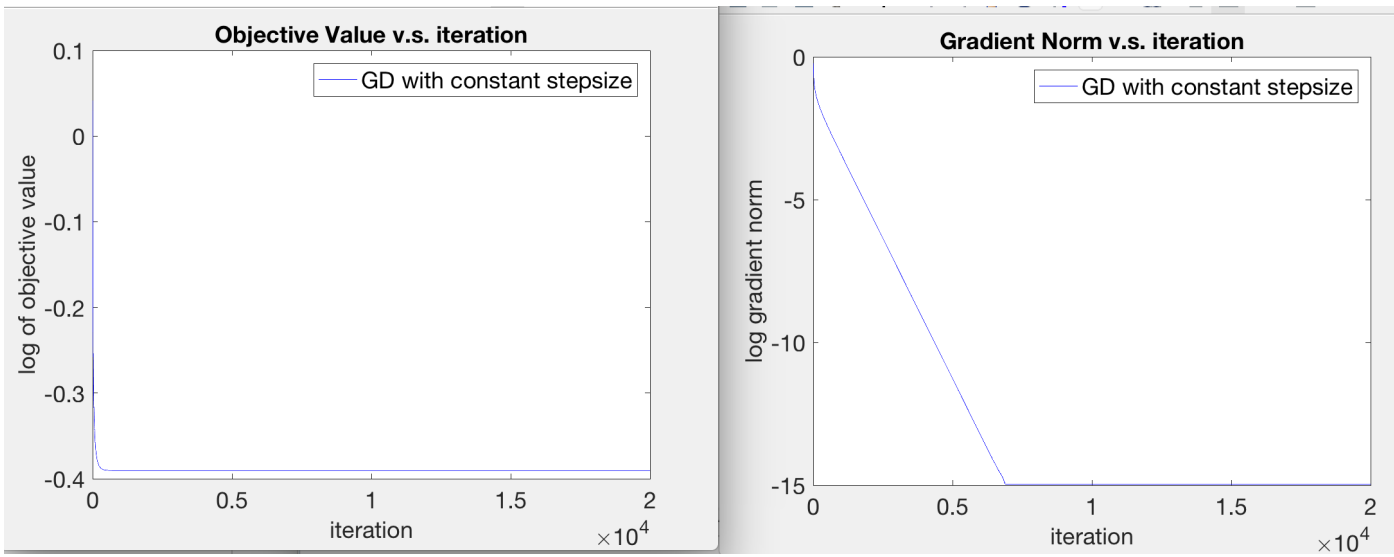
Summary of Results for 1-dim Logistic Regression

- Since $\log(1 + e^{-z})$ is strictly convex, but NOT strongly convex, the worst-case convergence rate is sublinear, i.e., $O(1/r)$.
- Two convergence patterns:
  - Separable case: sublinear rate $O(1/r)$
  - Non-separable case: usually linear rate $O(\rho^r)$.

Extension: Higher-Dimensional Case?

Data Setting 1: $n = 10, d = 5$.

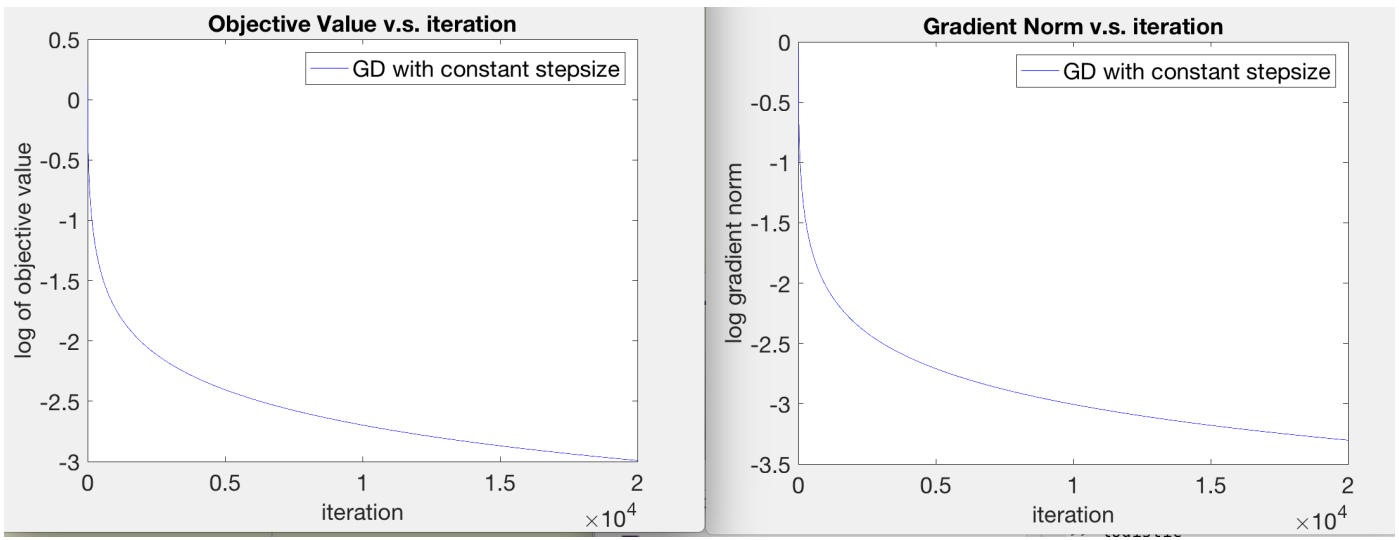
- Random Ganssian data $x_1, \ldots, x_{10} \in \mathbb{R}^5$.
- Random labels y_i with prob. $1/2$.



Extension: Higher-Dimensional Case?

Data Setting 2: $N = 10, d = 5$.

- Random Ganssian data $x_1, \ldots, x_{10} \in \mathbb{R}^5$.
- True separator $w^* = (1, 1, 1, 1, 1)$
- $y_i = \text{sign}(w' x_i), \forall i$.



Explain?

Applying Theory to Most Details

If you want, we could explain most details of the convergence behavior of LR.

e.g. in Data Setting 1, what determines the rate of convergence?

e.g. in Data Setting 2, it converges in $O(1/r)$ rate, what is the constant?

In higher-dimensional case, like $n, d = 1000$ or higher, the behavior may be more complicated.

- You are welcome to try

Extension: High-Dimensional Case?

Separable case causes convergence issues – this was known for a long time.

- Allison 2008, "Convergence Failures in Logistic Regression."
- "A frequent problem in estimating logistic regression models is a failure of the ... algorithm to converge."
  - Here "failure of converge" means "failure of iterates to converge"
  - We know that the function values will converge
- Two possible data patterns: complete separation, or quasi-complete separation. The second is very common.
- While complete separation is less common, combining LR with deep learning, it becomes more common

Resolve Issues in Separable Case

This was discussed in detail in [Allison 2008].

- Some statistical aspects need to be addressed

We discuss two possible solutions and the related issues.

Solution 1: Add regularizer, e.g., $\lambda\|\mathbf{w}\|^2$.

- Claim: If f is convex, then $f(\mathbf{w}) + \lambda\|\mathbf{w}\|^2$ is strongly convex.
- Issues:

Resolve Issues in Separable Case

This was discussed in detail in [Allison 2008].

- Some statistical aspects need to be addressed

We discuss two possible solutions and the related issues.

Solution 1: Add regularizer, e.g., $\lambda\|\mathbf{w}\|^2$.

- Claim: If f is convex, then $f(\mathbf{w}) + \lambda\|\mathbf{w}\|^2$ is strongly convex.
- Issues: how to tune λ? Distort original solutions? Still slow....

Solution 2: Forget $f(\mathbf{w})$, only care about classification error.

- More precisely, $\sum_i |y_i - \text{sign}(w_i' x_i)|$; if zero, stop

Conclusion of This Lecture

Can you summarize yourself?

Conclusion of This Lecture

Can you summarize yourself?

- Logistic regression problem may exhibit two convergence behavior: linear rate and sublinear rate
- When the data is good (separable), the problem is "hard"!
- Stopping criteria: $\|\nabla f(w^r)\|$ is a better metric for detecting convergence
- Sublinear rate: can happen even for 1-dim convex problems

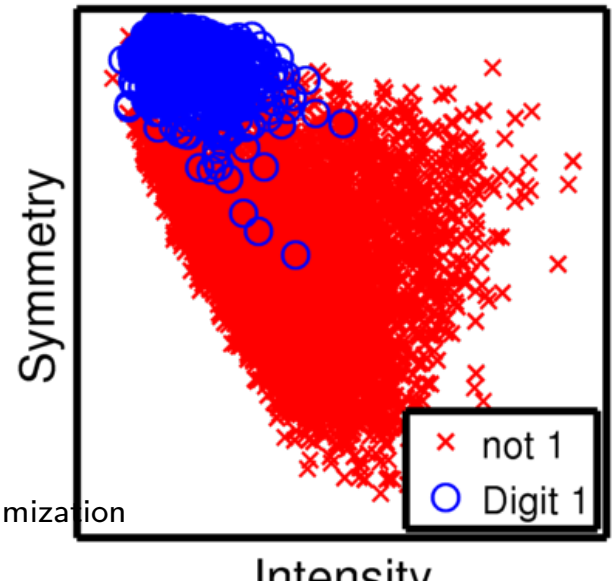
Homework 3

- Try out the algorithms on the following handwritten digit dataset.
- A sample of digits from the US postal Service Zip Code Database
- 16×16 pixel images
- **Objective** recognize the digit in each image

Figure 0.1: The MINST database

Homework 3

- **Features**: two dimensional, Symmetry and Average Intensity
- File: features_test.txt, features_train.txt
- Three columns, first column digit, second, third are features
- **Sample task**: separating the digit "1" with the rest, separating the digit "1" with "5" and so on
- Plot the separating line (the model) generated by different algorithms
- Some tutorial on reading and processing .tex files using MATLAB Tutorial 1 Tutorial 2 Tutorial 3



Homework 3 Sample Plot 1

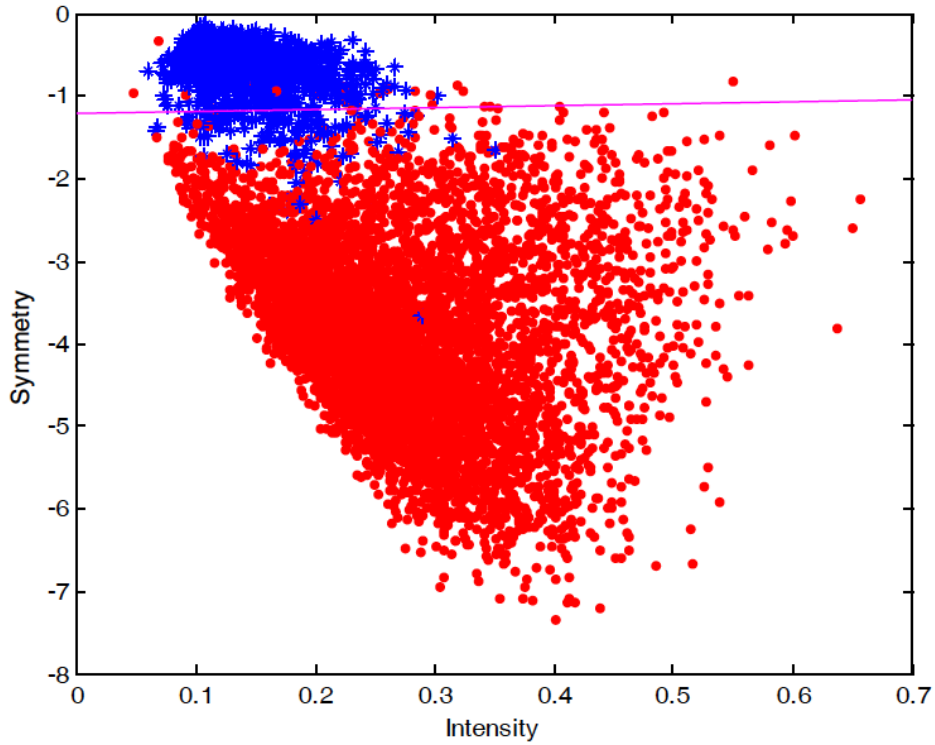


Figure 0.3: The plot of the separating line

Homework 3 Sample Plot 2

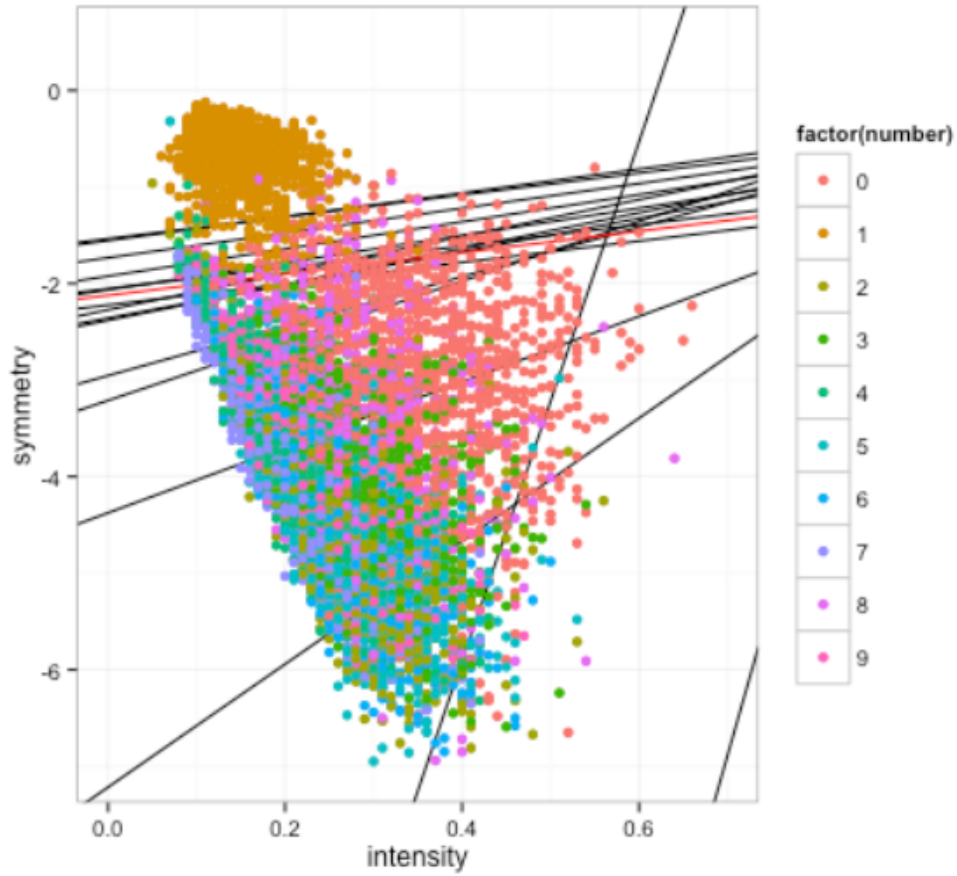


Figure 0.4: The plot of the separating line

Homework 3 Sample Plot 2

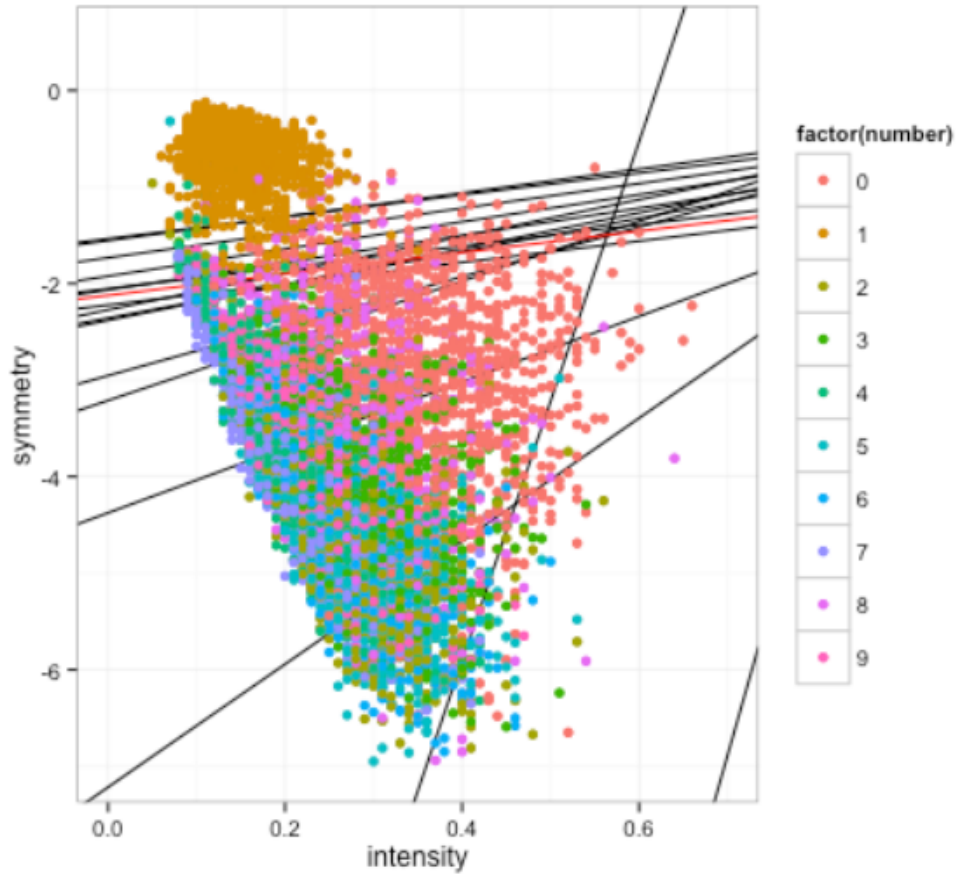


Figure 0.5: The plot of the separating line

Homework 3 Sample Plot 2

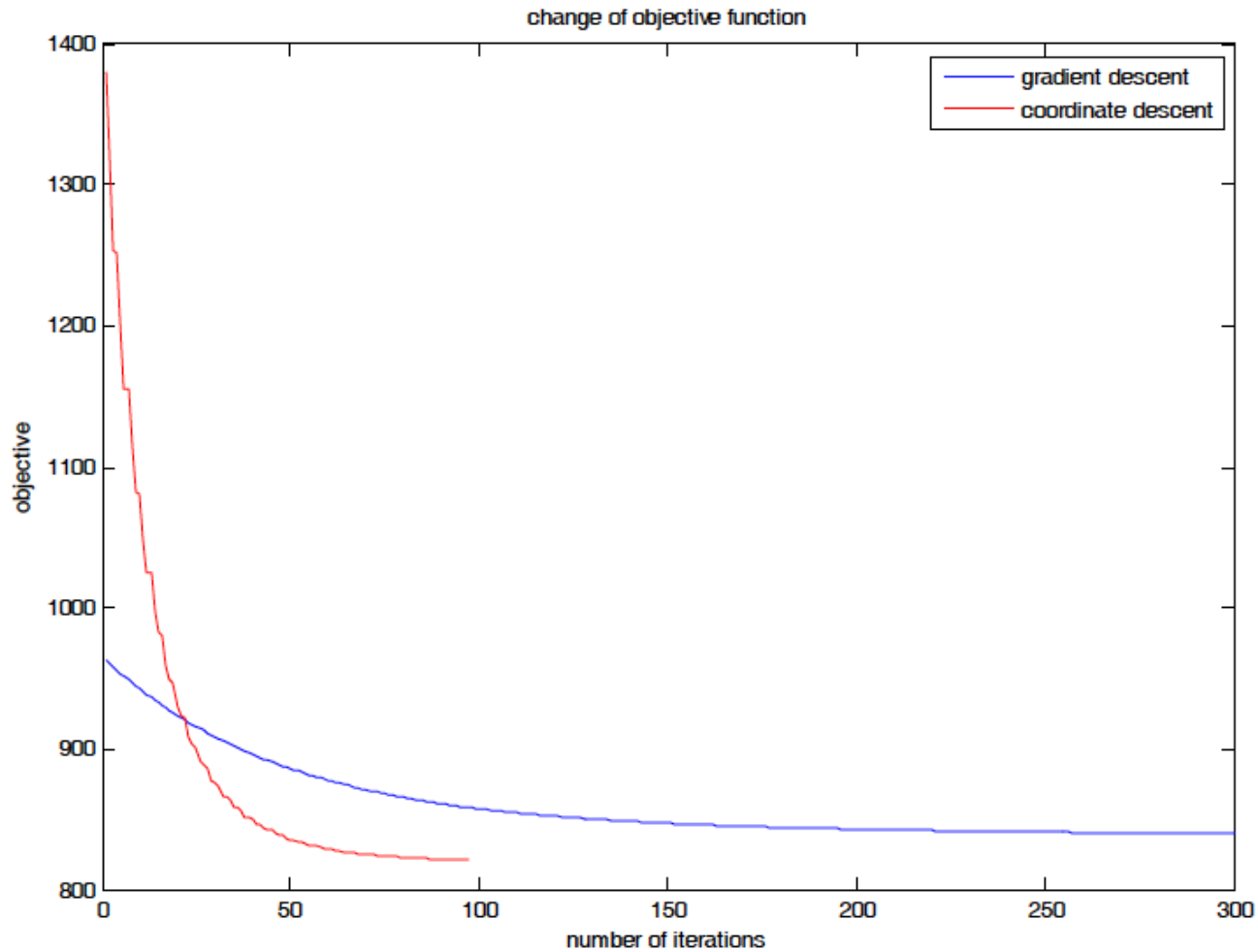


Figure 0.6: The plot of the separating line

Homework 3 Confusion Matrix

Error Rate	Is One	Not One
Is One	1709	21
Not One	34	243

Figure 0.7: The Error Matrix (either testing or the training errors)